Implement NAT Gateway Setup

Description

- **i.** Create two instances, one in a public subnet and one in a private subnet
- **ii.** Host a website inside public web server
- **iii.** Host a database inside private database server
- **iv.** Connect to DB server only from its webserver, shouldn't have a public access to server even if the security group has open entries
- **v.** DB Server should have internet access. Try to update the software packages, patching, etc.

1. Architecture Overview

This framework provides a secure, production-ready environment inside Amazon Web Services (AWS) using a custom Virtual Private Cloud (VPC). The deployment features an isolated front-end web application (WordPress) tier accessible over the public internet and a secure backend database tier (MariaDB) isolated from direct external access.

Network Topology & Parameters

- **VPC CIDR:** 10.0.0.0/16
- **Public Subnet:** 10.0.1.0/24 (Associated with Internet Gateway for external routing)
- **Private Subnet:** 10.0.3.0/24 (Associated with NAT Gateway for secure outbound-only updates)
- **Web Server EC2:** wordpress-web (Public IP: 13.201.44.26, Port 80/443 open to all)
- **Database Server EC2:** db-server (Private IP: 10.0.3.157, Port 3306 open ONLY to Web Server SG)

2. Core Infrastructure & Networking Setup

Follow these precise configuration steps inside the AWS Management Console or AWS CLI to prepare the network layer.

Step 1: Create Custom VPC

Provision a dedicated virtual network to isolate the environment resources:

```
VPC Name: Custom-VPC
CIDR Block: 10.0.0.0/16
```

Step 2: Provision Subnets

Divide the allocated CIDR block into separate zones for public and private infrastructure tiers:

```
Public Subnet:  10.0.1.0/24
Private Subnet: 10.0.3.0/24
```

Step 3: Deploy Internet Gateway (IGW)

To allow public internet ingress/egress to the public tier, provision and attach an Internet Gateway:

```
Action: Create Internet Gateway -> Attach to Custom-VPC
```

Step 4: Configure Public Route Table

Map traffic destined for the external internet through the newly deployed Internet Gateway:

```
Destination: 0.0.0.0/0
Target: Internet Gateway (igw-xxxxxx)
Association: Public Subnet (10.0.1.0/24)
```

3. Compute Instances & Security Group Hardening

Step 5: Launch WordPress Web Server

- Instance Name: wordpress-web
- Subnet Placement: Public Subnet (10.0.1.0/24)
- Public IP Assignment: Enabled

Define Security Group Rules (web-sg):

```
Inbound Rules:
- SSH (Port 22) -> Restrained to [Your Admin IP Account Only]
- HTTP (Port 80) -> Allow from Anywhere (0.0.0.0/0)
- HTTPS (Port 443) -> Allow from Anywhere (0.0.0.0/0)
```

Step 6: Launch Private Database Server

- Instance Name: db-server
- Subnet Placement: Private Subnet (10.0.3.0/24)
- Public IP Assignment: Disabled (Completely Private)

Define Security Group Rules (db-sg) to secure the internal perimeter:

```
Inbound Rules:
- SSH (Port 22) -> Restrained to web-sg Source Group Only
- MYSQL/MariaDB (Port 3306) -> Restrained to web-sg Source Group Only
```

> *Security Design Matrix: Even if malicious entries are erroneously added to open up permissions publicly, the db-server lacks a public IPv4 configuration and resides in a subnet with no route to the Internet Gateway, making it completely unreachable from the public internet.*

4. Outbound Internet via NAT Gateway

To fulfill package updates, dependency resolutions, and security patching on the backend database without breaking isolation protocols, implement an AWS NAT Gateway.

Step 7: Allocate Elastic IP & Provision NAT Gateway

Allocate a static public Elastic IP and anchor the NAT Gateway inside the public perimeter so it can route transactions:

```
1. Allocate standard AWS Elastic IP (EIP)
2. Provision NAT Gateway inside the Public Subnet (10.0.1.0/24)
3. Map the allocated EIP to this NAT Gateway
```

Step 8: Configure Private Route Table

Instruct the private subnet to delegate all external requests directly through the NAT instance:

```
Destination: 0.0.0.0/0
Target: NAT Gateway (nat-xxxxxx)
Association: Private Subnet (10.0.3.0/24)
```

Step 9: Verify Secure NAT Egress Connectivity

Connect to the instance via jump host configuration and test software repository interactions using the system utility:

```
sudo dnf update -y
```

5. Key Management & Server Interconnectivity

Step 10: Secure Deployment Key Transfer

Safely staged deployment credentials by passing the administrative key material from the provisioning workstation down to the designated management jump point (Web Server):

```
From Local Workstation:
scp -i internship-key.pem internship-key.pem ec2-user@13.201.44.26:/home/ec2-user/

Establish Management Session:
ssh -i internship-key.pem ec2-user@13.201.44.26

Apply POSIX Standard Minimum File Permissions:
chmod 400 /home/ec2-user/internship-key.pem
```

Step 11: Pivot Access to Private Database Infrastructure

Utilize internal horizontal network routing pathing from the Web Server to land inside the secure database tier:

```
ssh -i /home/ec2-user/internship-key.pem ec2-user@10.0.3.157
```

6. Database Tier Installation & Configuration

With connectivity confirmed, configure the backend MariaDB installation natively inside the isolated private layer.

Step 12: Install and Modernize the MariaDB Stack

```
# Download and install core database server packages
dnf install mariadb-server -y

# Enable service configuration initialization at runtime bootstrap
systemctl enable --now mariadb

# Execute standardized environment interactive security scripts
mysql_secure_installation
```

Step 13: Create Application Database Schema and Access Constraints

Access the database engine locally and allocate a structured zone restricted precisely to the front-end network subnet segment (10.0.1.%):

```
mysql -u root -p

-- Database and Scope Provisioning Queries
CREATE DATABASE wordpress;
CREATE USER 'wpuser'@'10.0.1.%' IDENTIFIED BY 'StrongPassword123';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'10.0.1.%';
FLUSH PRIVILEGES;
SHOW DATABASES;
```

Step 14: Verify Port Interception and Binding Profiles

Confirm that the database application layer is successfully processing internal network events across the target port:

```
sudo ss -tulpn | grep 3306

# Expected Status Verification Response Matrix:
# 0.0.0.0:3306 (Listening across all localized adapter instances)
```

Step 15: Cross-Tier Ingress Validation Validation

Pivot back to the public web server instance and assert direct connection parameters to the newly listening database asset:

```
mysql -h 10.0.3.157 -u wpuser -p wordpress

# Expected Output Confirmation:
# MariaDB [wordpress]>
```

7. Web Application Deployment & Environment Hardening

Execute system-wide server tasks to deploy the front-facing production environment.

Step 16: Setup Apache Web Stack and PHP Modules

```
dnf install httpd php php-mysqlnd php-gd php-xml php-mbstring wget tar -y
systemctl enable --now httpd
```

Step 17: Download and Stage Web Assets

```
cd /var/www/html
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
```

```
cp -r wordpress/* .
rm -rf wordpress latest.tar.gz
```

Step 18: Wire Web Settings to Private Core Database

Customize application configuration settings to reference internal system targets directly:

```
cp wp-config-sample.php wp-config.php
vi wp-config.php

# Configure Parameters:
define('DB_NAME', 'wordpress');
define('DB_USER', 'wpuser');
define('DB_PASSWORD', 'StrongPassword123');
define('DB_HOST', '10.0.3.157');
```

Step 19: Rectify Operating System Discretionary Permissions

Enforce ownership standards across web asset blocks to align cleanly with system runtime specifications:

```
chown -R apache:apache /var/www/html
chmod -R 755 /var/www/html
```

Step 20: Reload Active Runtime Daemons

```
systemctl restart httpd
```

8. Advanced Security Diagnostics (SELinux Mitigation)

Step 21: Resolve Common Database Connection Interceptions

If encountering standard systemic failures such as 'Error establishing database connection', systematically isolate network blocks and inspect mandatory system control policies:

```
# 1. Audit core configuration variables
grep DB_ wp-config.php

# 2. Test explicit socket interface reachability
mysql -h 10.0.3.157 -u wpuser -p wordpress

# 3. Inspect module load statuses
php -m | grep mysql
```

```
# 4. Check active Red Hat SELinux policy configurations for httpd network
rules
getsebool httpd_can_network_connect_db
```

Mandatory Policy Mitigation Action Plan:

```
# Modify SELinux boolean configuration properties persistently across reboots
setsebool -P httpd_can_network_connect_db on

# Recycle system daemon instances
systemctl restart httpd
```

9. Final Operational Verification Phases

Step 22: Complete Web Administrative Installation Wizard

Point a local secure web browser instance to the public cloud endpoint target to verify stack behavior:

```
URL Target Route: http://13.201.44.26
Required Fields: Site Title, Admin User, Password, Email -> Click Install
```

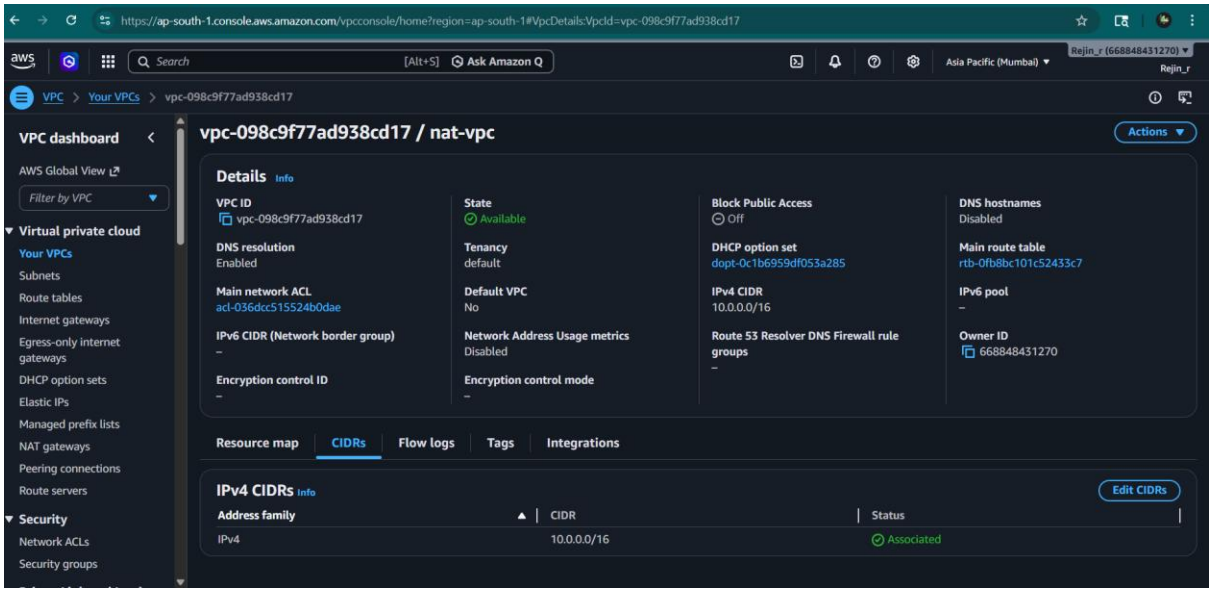
Step 23: Complete Backend Operational Assertions

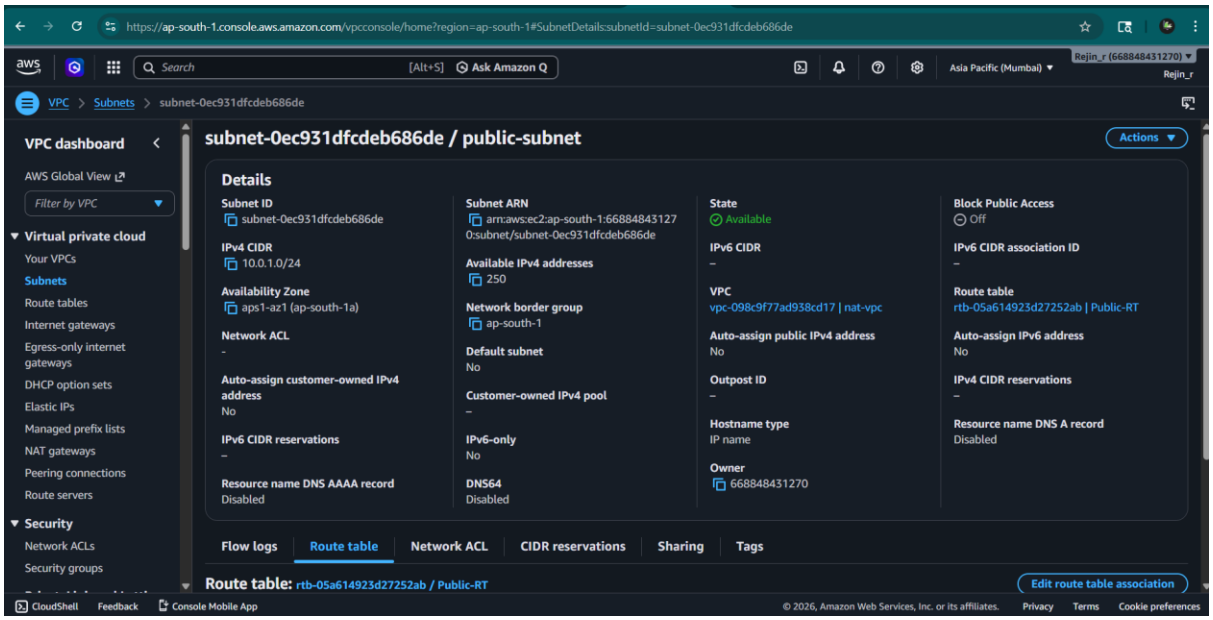
Access the isolated private data engine once more to ensure database records have populated successfully:

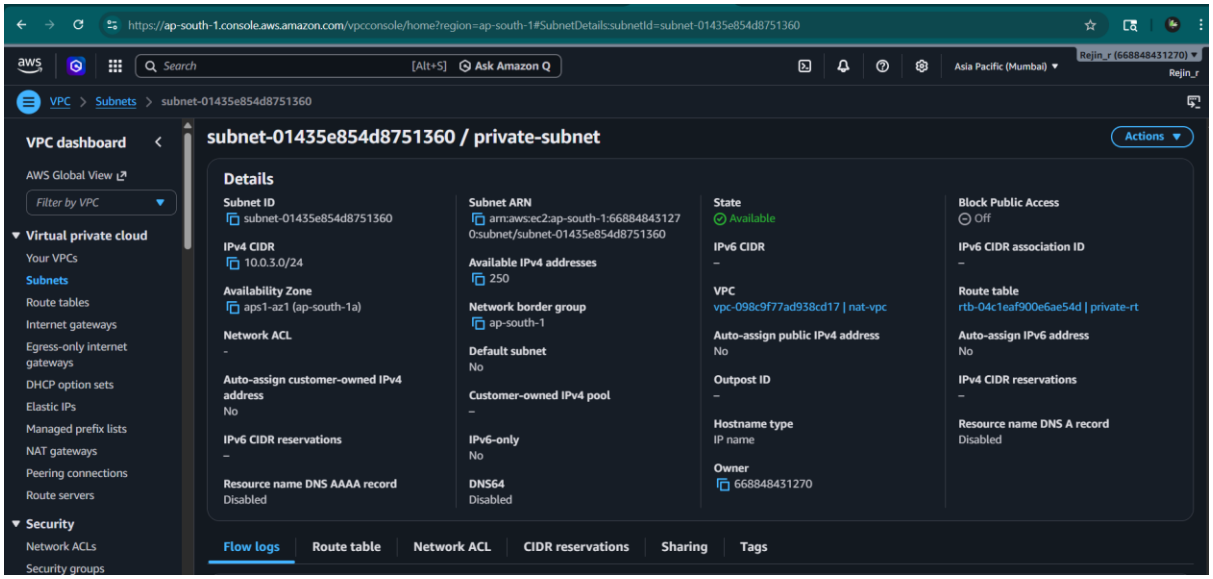
```
mysql -h 10.0.3.157 -u wpuser -p wordpress
SHOW TABLES;

# Confirmed Structural Schemas Generated:
# wp_users, wp_posts, wp_options, wp_comments, wp_terms
```

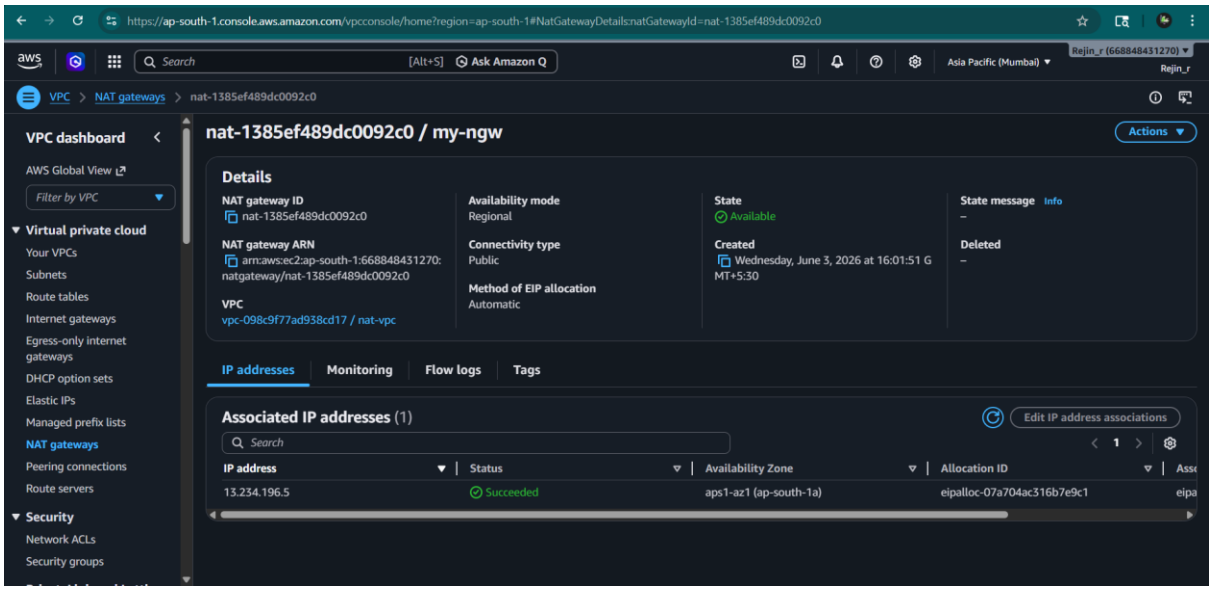
```
Normally, root should only be allowed to connect from 'localhost'.  This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] y
 ... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access.  This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] y
 - Dropping test database...
 ... Success!
 - Removing privileges on test database...
 ... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] y
 ... Success!

Cleaning up...

All done!  If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
[root@ip-10-0-3-64 ~]# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 19
Server version: 10.11.15-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE wordpress;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> CREATE USER 'wpuser'@'10.0.1.%' IDENTIFIED BY 'StrongPassword@123';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'10.0.1.%';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> EXIT;
Bye
[root@ip-10-0-3-64 ~]#
```

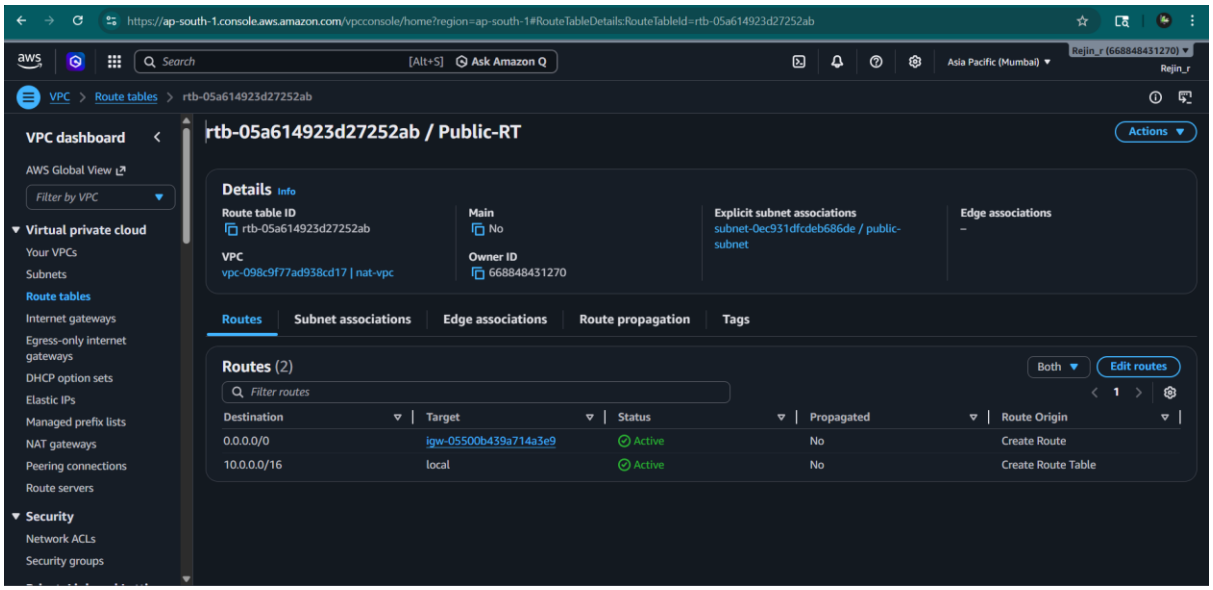
```
 * * Database table prefix
 * * ABSPATH
 *
 * @link https://developer.wordpress.org/advanced-administration/wordpress/wp-config/
 *
 * @package WordPress
 */

// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress' );

/** Database username */
define( 'DB_USER', 'wpuser' );

/** Database password */
define( 'DB_PASSWORD', 'StrongPassword@123' );

/** Database hostname */
define( 'DB_HOST', '10.0.3.64' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8mb4' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );

/**#@+
 * Authentication unique keys and salts.
 *
 * Change these to different unique phrases! You can generate these using
 * the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}.
 *
 * You can change these at any point in time to invalidate all existing cookies
```

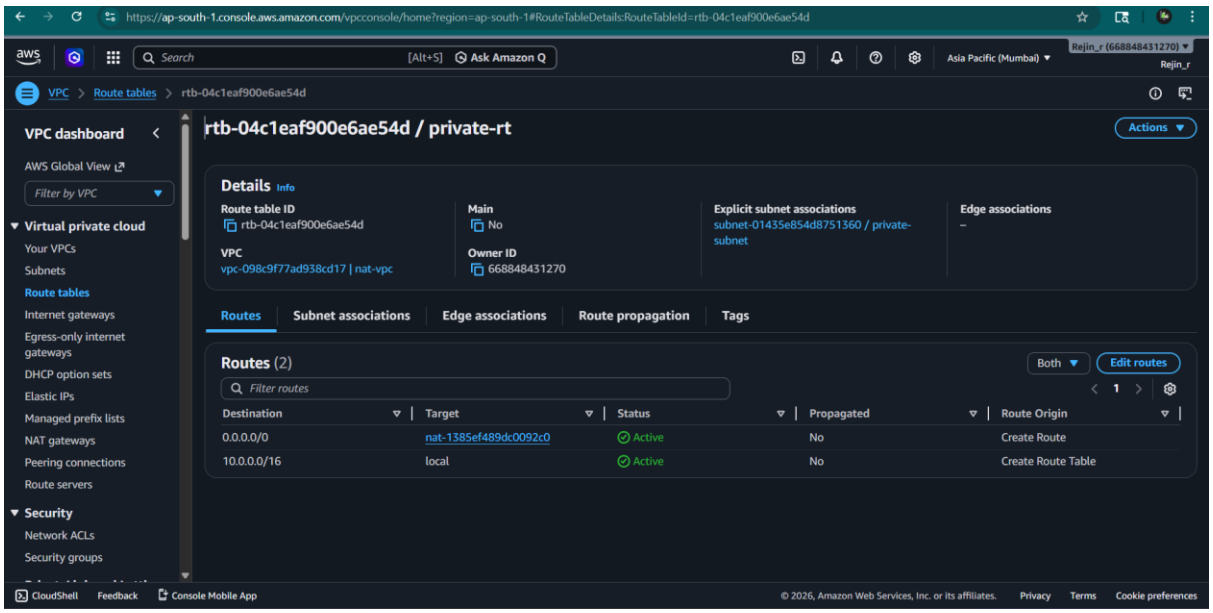

https://ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#VpcDetails:VpcId=vpc-098c9f77ad938cd17
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Rejin_r (668848431270)
Rejin_r
VPC > Your VPCs > vpc-098c9f77ad938cd17
VPC dashboard
AWS Global View
Filter by VPC
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
vpc-098c9f77ad938cd17 / nat-vpc
Actions
Details Info
VPC ID
vpc-098c9f77ad938cd17
State
Available
Block Public Access
Off
DNS hostnames
Disabled
DNS resolution
Enabled
Tenancy
default
DHCP option set
dopt-0c1b6959df053a285
Main route table
rtb-0fb8bc101c52433c7
Main network ACL
acl-036dcc515524b0dae
Default VPC
No
IPv4 CIDR
10.0.0.0/16
IPv6 pool
–
IPv6 CIDR (Network border group)
–
Network Address Usage metrics
Disabled
Route 53 Resolver DNS Firewall rule groups
–
Owner ID
668848431270
Encryption control ID
–
Encryption control mode
–
Resource map
CIDRs
Flow logs
Tags
Integrations
IPv4 CIDRs Info
Edit CIDRs
Address family
CIDR
Status
IPv4
10.0.0.0/16
Associated

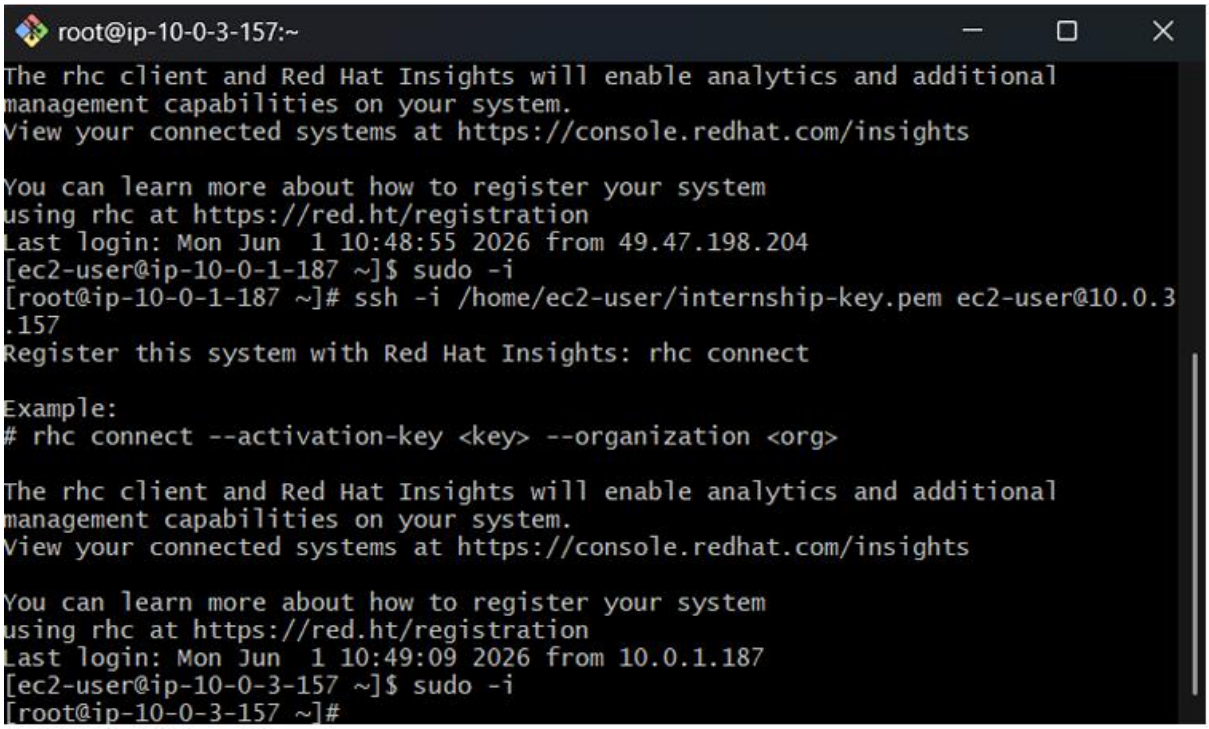

https://ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#SubnetDetails:subnetId=subnet-0ec931dfcdeb686de
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Rejin_r (668848431270)
Rejin_r
VPC > Subnets > subnet-0ec931dfcdeb686de
VPC dashboard
AWS Global View
Filter by VPC
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
subnet-0ec931dfcdeb686de / public-subnet
Actions
Details
Subnet ID
subnet-0ec931dfcdeb686de
Subnet ARN
arn:aws:ec2:ap-south-1:668848431270:subnet/subnet-0ec931dfcdeb686de
State
Available
Block Public Access
Off
IPv4 CIDR
10.0.1.0/24
Available IPv4 addresses
250
IPv6 CIDR
–
IPv6 CIDR association ID
–
Availability Zone
aps1-az1 (ap-south-1a)
Network border group
ap-south-1
VPC
vpc-098c9f77ad938cd17 | nat-vpc
Route table
rtb-05a614923d27252ab | Public-RT
Network ACL
-
Default subnet
No
Auto-assign public IPv4 address
No
Auto-assign IPv6 address
No
Auto-assign customer-owned IPv4 address
No
Customer-owned IPv4 pool
–
Outpost ID
–
IPv4 CIDR reservations
–
IPv6 CIDR reservations
–
IPv6-only
No
Hostname type
IP name
Resource name DNS A record
Disabled
Resource name DNS AAAA record
Disabled
DNS64
Disabled
Owner
668848431270
Flow logs
Route table
Network ACL
CIDR reservations
Sharing
Tags
Route table: rtb-05a614923d27252ab / Public-RT
Edit route table association
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

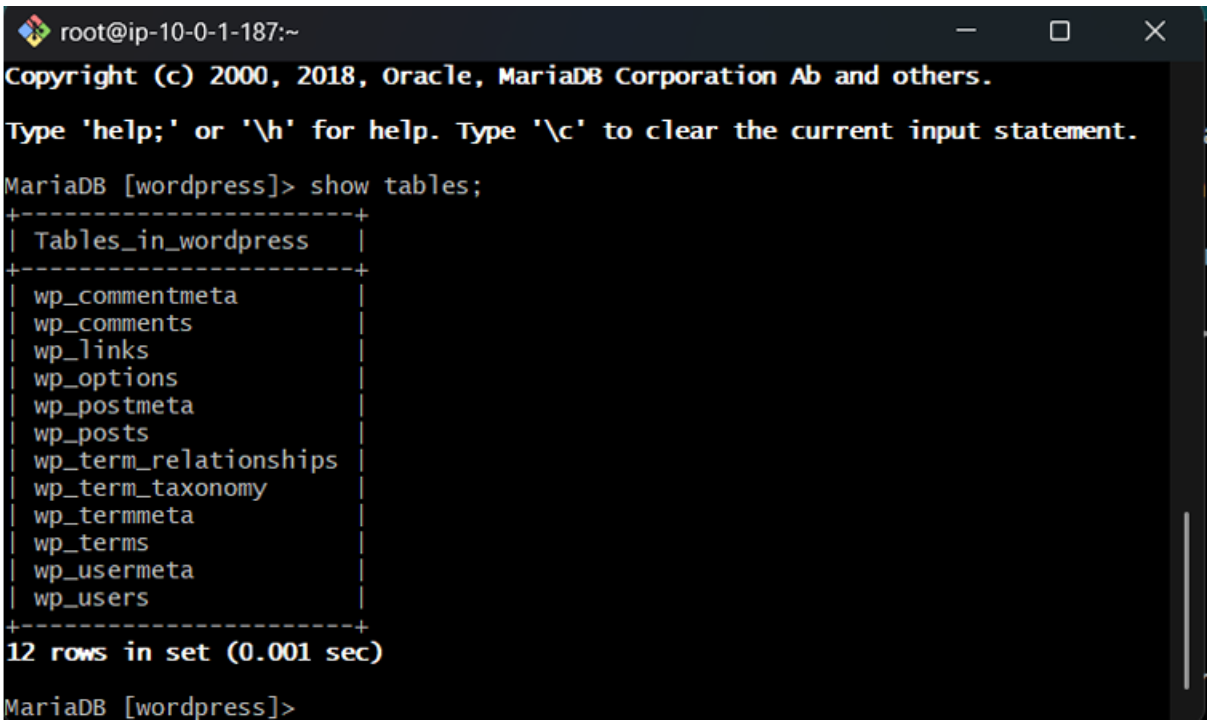

https://ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#SubnetDetails:subnetId=subnet-01435e854d8751360
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Rejin_r (668848431270)
Rejin_r
VPC > Subnets > subnet-01435e854d8751360
VPC dashboard
AWS Global View
Filter by VPC
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
subnet-01435e854d8751360 / private-subnet
Actions
Details
Subnet ID
subnet-01435e854d8751360
Subnet ARN
arn:aws:ec2:ap-south-1:668848431270:subnet/subnet-01435e854d8751360
State
Available
Block Public Access
Off
IPv4 CIDR
10.0.3.0/24
Available IPv4 addresses
250
IPv6 CIDR
–
IPv6 CIDR association ID
–
Availability Zone
aps1-az1 (ap-south-1a)
Network border group
ap-south-1
VPC
vpc-098c9f77ad938cd17 | nat-vpc
Route table
rtb-04c1eaf900e6ae54d | private-rt
Network ACL
-
Default subnet
No
Auto-assign public IPv4 address
No
Auto-assign IPv6 address
No
Auto-assign customer-owned IPv4 address
No
Customer-owned IPv4 pool
–
Outpost ID
–
IPv4 CIDR reservations
–
IPv6 CIDR reservations
–
IPv6-only
No
Hostname type
IP name
Resource name DNS A record
Disabled
Resource name DNS AAAA record
Disabled
DNS64
Disabled
Owner
668848431270
Flow logs
Route table
Network ACL
CIDR reservations
Sharing
Tags

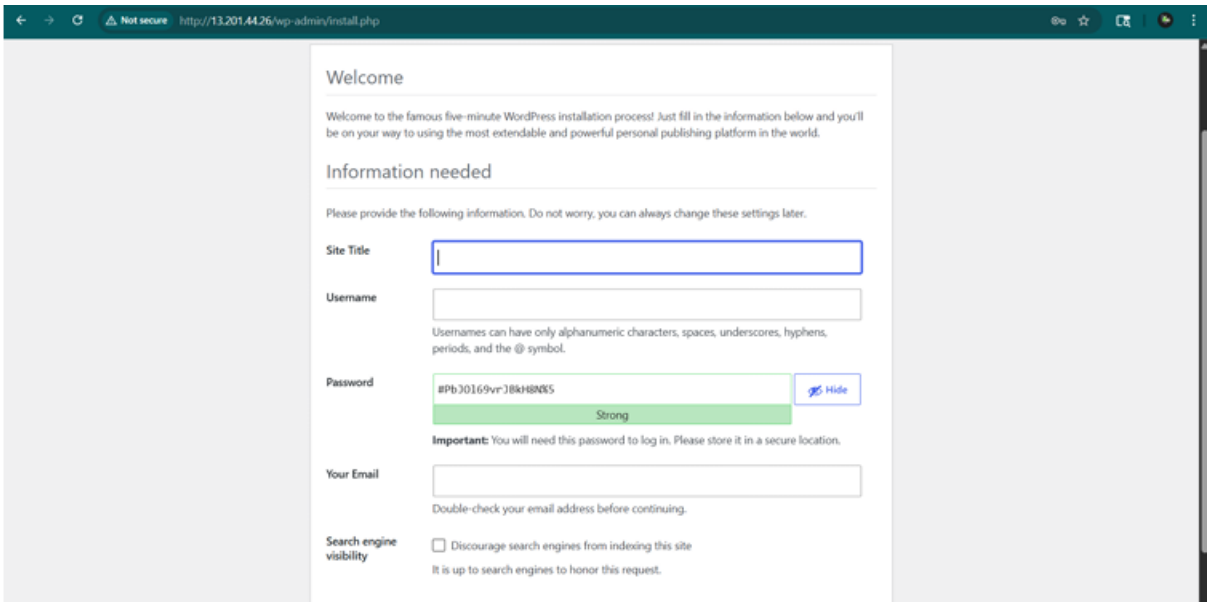

https://ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#NatGatewayDetails:natGatewayId=nat-1385ef489dc0092c0
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Rejin_r (668848431270)
Rejin_r
VPC > NAT gateways > nat-1385ef489dc0092c0
VPC dashboard
AWS Global View
Filter by VPC
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
nat-1385ef489dc0092c0 / my-ngw
Actions
Details
NAT gateway ID
nat-1385ef489dc0092c0
Availability mode
Regional
State
Available
State message
Info
–
NAT gateway ARN
arn:aws:ec2:ap-south-1:668848431270:natgateway/nat-1385ef489dc0092c0
Connectivity type
Public
Created
Wednesday, June 3, 2026 at 16:01:51 GMT+5:30
Deleted
–
VPC
vpc-098c9f77ad938cd17 / nat-vpc
Method of EIP allocation
Automatic
IP addresses
Monitoring
Flow logs
Tags
Associated IP addresses (1)
Edit IP address associations
Search
IP address
Status
Availability Zone
Allocation ID
13.234.196.5
Succeeded
aps1-az1 (ap-south-1a)
eipalloc-07a704ac316b7e9c1

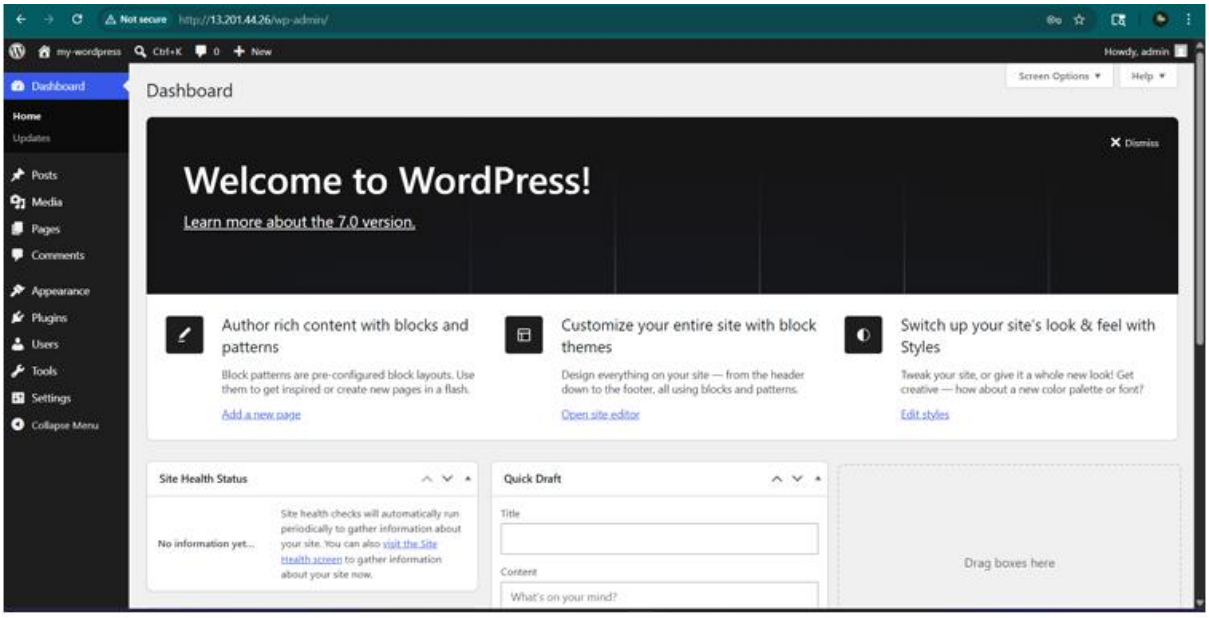

https://ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#RouteTableDetails:RouteTableId=rtb-05a614923d27252ab
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Rejin_r (668848431270)
Rejin_r
VPC > Route tables > rtb-05a614923d27252ab
VPC dashboard
AWS Global View
Filter by VPC
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
rtb-05a614923d27252ab / Public-RT
Actions
Details Info
Route table ID
rtb-05a614923d27252ab
Main
No
Explicit subnet associations
subnet-0ec931dfcdeb686de / public-subnet
Edge associations
–
VPC
vpc-098c9f77ad938cd17 | nat-vpc
Owner ID
668848431270
Routes
Subnet associations
Edge associations
Route propagation
Tags
Routes (2)
Both
Edit routes
Filter routes
1
Destination
Target
Status
Propagated
Route Origin
0.0.0.0/0
igw-05500b439a714a3e9
Active
No
Create Route
10.0.0.0/16
local
Active
No
Create Route Table


https://ap-south-1.console.aws.amazon.com/vpcconsole/home?region=ap-south-1#RouteTableDetails:RouteTableId=rtb-04c1eaf900e6ae54d
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Rejin_r (668848431270)
Rejin_r
VPC > Route tables > rtb-04c1eaf900e6ae54d
VPC dashboard
AWS Global View
Filter by VPC
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
rtb-04c1eaf900e6ae54d / private-rt
Actions
Details Info
Route table ID
rtb-04c1eaf900e6ae54d
Main
No
Explicit subnet associations
subnet-01435e854d8751360 / private-subnet
Edge associations
–
VPC
vpc-098c9f77ad938cd17 | nat-vpc
Owner ID
668848431270
Routes
Subnet associations
Edge associations
Route propagation
Tags
Routes (2)
Both
Edit routes
Filter routes
1
Destination
Target
Status
Propagated
Route Origin
0.0.0.0/0
nat-1385ef489dc0092c0
Active
No
Create Route
10.0.0.0/16
local
Active
No
Create Route Table
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


root@ip-10-0-3-157:~
The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights
You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Mon Jun 1 10:48:55 2026 from 49.47.198.204
[ec2-user@ip-10-0-1-187 ~]$ sudo -i
[root@ip-10-0-1-187 ~]# ssh -i /home/ec2-user/internship-key.pem ec2-user@10.0.3
.157
Register this system with Red Hat Insights: rhc connect
Example:
rhc connect --activation-key <key> --organization <org>
The rhc client and Red Hat Insights will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights
You can learn more about how to register your system
using rhc at https://red.ht/registration
Last login: Mon Jun 1 10:49:09 2026 from 10.0.1.187
[ec2-user@ip-10-0-3-157 ~]$ sudo -i
[root@ip-10-0-3-157 ~]#


root@ip-10-0-1-187:~
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
MariaDB [wordpress]> show tables;
| Tables_in_wordpress |
| wp_commentmeta |
| wp_comments |
| wp_links |
| wp_options |
| wp_postmeta |
| wp_posts |
| wp_term_relationships |
| wp_term_taxonomy |
| wp_termmeta |
| wp_terms |
| wp_usermeta |
| wp_users |
12 rows in set (0.001 sec)
MariaDB [wordpress]>


Not secure http://13.201.44.26/wp-admin/install.php
Welcome
Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.
Information needed
Please provide the following information. Do not worry, you can always change these settings later.
Site Title
Username
Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.
Password
#Pb3O169vrJ8kH8N%5
Hide
Strong
Important: You will need this password to log in. Please store it in a secure location.
Your Email
Double-check your email address before continuing.
Search engine visibility
Discourage search engines from indexing this site
It is up to search engines to honor this request.


Not secure http://13.201.44.26/wp-admin/
my-wordpress
Ctrl+K
0
New
Howdy, admin
Dashboard
Home
Updates
Posts
Media
Pages
Comments
Appearance
Plugins
Users
Tools
Settings
Collapse Menu
Screen Options
Help
Welcome to WordPress!
Learn more about the 7.0 version.
Dismiss
Author rich content with blocks and patterns
Block patterns are pre-configured block layouts. Use them to get inspired or create new pages in a flash.
Add a new page
Customize your entire site with block themes
Design everything on your site — from the header down to the footer, all using blocks and patterns.
Open site editor
Switch up your site's look & feel with Styles
Tweak your site, or give it a whole new look! Get creative — how about a new color palette or font?
Edit styles
Site Health Status
No information yet...
Site health checks will automatically run periodically to gather information about your site. You can also visit the Site Health screen to gather information about your site now.
Quick Draft
Title
Content
What's on your mind?
Drag boxes here